C++ ile Göstericiler (Pointers) ve Dizi Algoritmaları

Pointer Aritmetiği, Sıralama ve Arama İşlemleri

Prof.Dr.Kasım KURT

Fizik Bölümü

Ders İçeriği

1. Temel Kavramlar: Pointer Aritmetiği
2. Dizideki En Büyük Elemanı Bulma
3. Pointer ile Dizi Sıralama (Bubble Sort)
4. Örnek Program ve Çıktı

Diziler ve Gösterici (Pointer) İlişkisi

- C++ dilinde bir dizi adı, dizinin ilk elemanının bellek adresine işaret eden sabit bir göstericidir (`pointer`).
- `arr[i]` ifadesi, derleyici seviyesinde doğrudan işaretçi aritmetiğine dönüştürülür:

$$\texttt{arr[i]} \iff \texttt{*(arr + i)}$$

- **Adres Öteleme:** `arr + 1` adresi, dizinin veri tipinin bayt boyutu kadar (`sizeof(int)`) ileri ötelenir.

Önemli Kural

Gösterici aritmetiğinde `*arr + 1` ile `*(arr + 1)` farklıdır. Parantez kullanımı işlem önceliği açısından zorunludur.

En Büyük Elemanı Bulma (Değer Döndüren Yöntem)

```cpp
// Dizideki en buyuk degeri donduren fonksiyon
int enBuyukElemaniBul(const int* arr, int boyut) {
    if (arr == nullptr || boyut <= 0) return -1;

    int enBuyuk = *arr; // Ilk eleman (arr[0])

    for (int i = 1; i < boyut; ++i) {
        // *(arr + i) ile sonraki adreslerdeki degerler okunur
        if (*(arr + i) > enBuyuk) {
            enBuyuk = *(arr + i);
        }
    }
    return enBuyuk;
}
```

- `const int*`: Dizi içeriğinin kazara değiştirilmesini derleme aşamasında engeller (Salt okunur).
- `*arr`: Dizinin ilk elemanını başlangıç referansı olarak alır.

En Büyük Elemanı Bulma (Adres Döndüren Alternatif)

```cpp
// En buyuk elemanin dogrudan bellek adresini dondurur
const int* enBuyukAdresBul(const int* arr, int boyut) {
    if (arr == nullptr || boyut <= 0) return nullptr;

    const int* maxPtr = arr; // Baslangic gostericisi

    for (const int* ptr = arr + 1; ptr < arr + boyut; ++ptr) {
        if (*ptr > *maxPtr) {
            maxPtr = ptr;
        }
    }
    return maxPtr; // Adres dondurulur
}

```

Adres Farkı ile İndeks Hesabı:

- `int indeks = maxPtr - arr;` formülü ile iki adresin farkı alınarak doğrudan indeks değeri elde edilebilir.

Sıralama Fonksiyonu (Bubble Sort)

```
1  // Pointer aritmetigi ile kucukten buyuge siralama
2  void diziSirala(int* arr, int boyut) {
3      if (arr == nullptr || boyut <= 1) return;
4
5      for (int i = 0; i < boyut - 1; ++i) {
6          for (int j = 0; j < boyut - i - 1; ++j) {
7              // *(arr + j) ifadesi arr[j] karsiligidir
8              if (*(arr + j) > *(arr + j + 1)) {
9                  // Bellekteki iki degeri takas etme (swap)
10                 int temp = *(arr + j);
11                 *(arr + j) = *(arr + j + 1);
12                 *(arr + j + 1) = temp;
13             }
14         }
15     }
16 }
17
```

Sıralama Algoritmasının Çalışma Mantığı

Algoritma Adımları:

1. Dizi başlangıç göstericisi (`arr`) fonksiyona geçirilir.
2. İç içe döngülerle ardışık elemanlar `*(arr + j)` ve `*(arr + j + 1)` kıyaslanır.
3. Sıra bozuksa yerel takas (`swap`) yapılır.
4. n elemanlı dizi için zaman karmaşıklığı $O(n^2)$ seviyesindedir.

Bellek Üzerindeki Etki

Fonksiyona dizinin bir kopyası değil, doğrudan adresi aktarıldığı için (*pass-by-pointer*) yapılan tüm yer değiştirmeler ana bellekteki orijinal dizi üzerinde gerçekleşir.

Ana Program (main.cpp)

```cpp
#include <iostream>
using namespace std;
int main() {
    int n = 5;
    int* dizi = new int[n]{45, 12, 89, 34, 23};

    cout << "Maksimum: " << enBuyukElemaniBul(dizi, n) << "\n";

    diziSirala(dizi, n);

    cout << "Sirali Dizi: ";
    for (int i = 0; i < n; ++i) {
       cout << *(dizi + i) << " ";
    }
    cout << "\n";

    delete[] dizi; // Bellek sizintisini onleme
    return 0;
}
```

Özet ve Dikkat Edilmesi Gerekenler

- **Sınır Kontrolleri:** Pointer aritmetiğinde sınır aşımı (*buffer overflow*) derleyici hatası üretmez; çalışma zamanı bellek bozulmalarına yol açar.
- **Null Kontrolü:** Pointer alan tüm fonksiyonlarda `nullptr` doğrulaması yapılmalıdır.
- **Bellek Yönetimi:** `new[]` ile dinamik açılan diziler mutlaka `delete[]` ile serbest bırakılmalıdır.
- **Const Doğruluğu:** Sadece okuma yapan fonksiyonlarda `const int*` parametre tipi tercih edilmelidir.